

satsim — Semana 1: o que foi desenvolvido

Oct 5, 2026 · @Renato

Resumo geral

A Semana 1 entregou a fundação do satsim: um núcleo de mecânica orbital em Python, validado contra os PDFs teóricos e o Skyfield, e um primeiro protótipo visual com globo 3D e mapa 2D animados. São 8 cards concluídos e 118 testes passando, com 1 falha esperada e documentada.

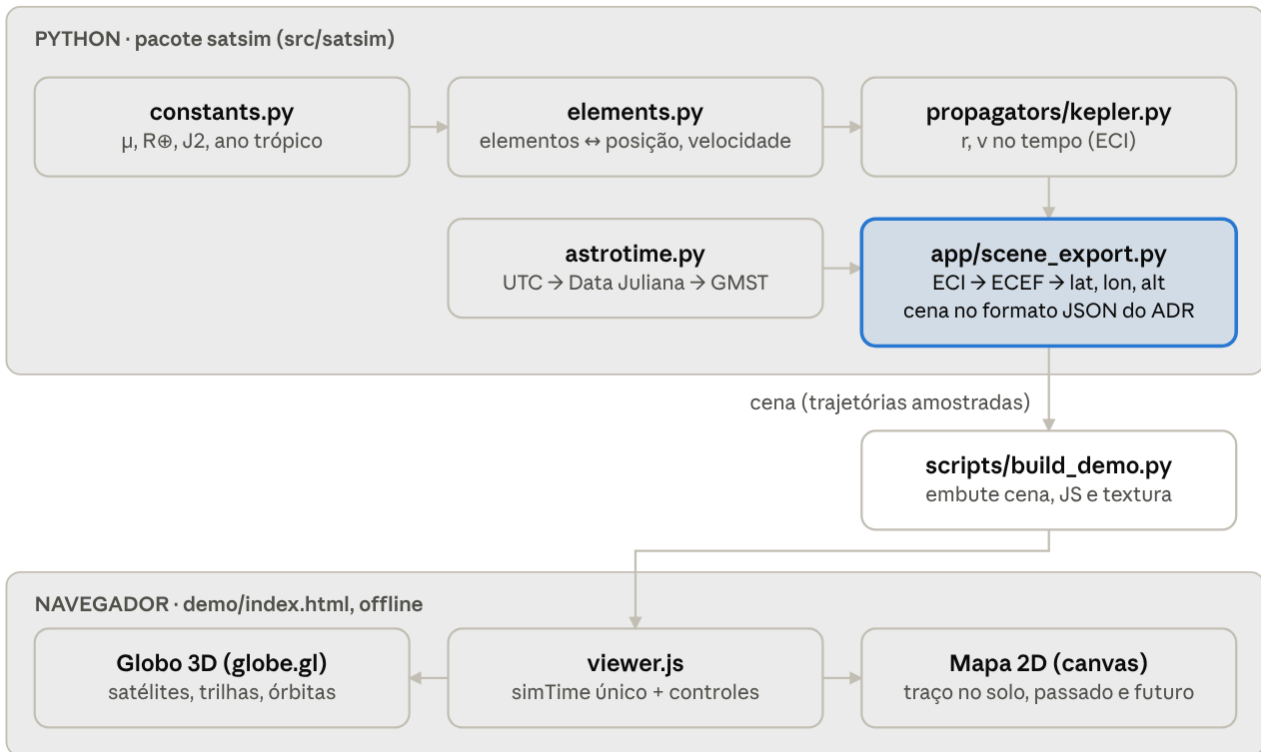
O **satsim** é um simulador de posicionamento de satélites. O plano tem três fases: a ISS sobre São Bento do Sapucaí (semanas 1–4), uma constelação heliossíncrona de 6 satélites (semana 5) e uma aplicação web com 18 satélites (semanas 6–7).

Três regras de arquitetura valem para todo o código:

- **Interface única de propagação:** todo propagador implementa `propagate(t) → (r, v)`, vetorizado. Trocar o modelo de órbita não muda o resto do código.
- **Multi-satélite desde o início:** adicionar satélites é só criar mais objetos; nenhuma análise assume um único satélite.
- **A física fica em Python; a interface só desenha:** o Python exporta a "cena" (trajetórias amostradas no tempo, em JSON) e o viewer HTML/JS apenas interpola e projeta.

O código está no repositório `satellite_constellation_simulation`, na branch `semana-1`, com um commit por card (mais ajustes de revisão). Convenções: unidades SI, ângulos em radianos internamente, tempo em UTC com fuso horário explícito.

O Python calcula a cena; o navegador só desenha



arquitetura da Semana 1 · do cálculo em Python à tela

Os módulos de cima formam o núcleo numérico; `scene_export.py` é a ponte, e tudo o que roda no navegador só lê a cena e desenha.

S1-01 · Setup do projeto

O repositório ganhou empacotamento, testes, lint e integração contínua: qualquer pessoa instala com um comando e o GitHub roda os testes a cada push.

Arquivo	O que faz
<code>pyproject.toml</code>	Empacotamento (hatchling) do pacote <code>satsim</code> 0.1.0; dependências <code>numpy</code> e <code>scipy</code> ; extra dev com <code>pytest</code> , <code>pytest-cov</code> , <code>ruff</code> e <code>skyfield</code> ; configuração do <code>pytest</code> e do <code>ruff</code>
<code>src/satsim/__init__.py</code>	Pacote principal, com a versão <code>__version__ = "0.1.0"</code>
<code>tests/test_smoke.py</code>	Teste de fumaça: o pacote importa e expõe a versão

Arquivo	O que faz
<code>.gitignore</code>	Ignora ambientes virtuais, caches, builds e a pasta <code>demo/gerada</code>
<code>README.md</code>	Instalação, comandos, estrutura, roadmap das 7 semanas, demo visual e golden tests
<code>.github/workflows/ci.yml</code>	CI no GitHub Actions: <code>ruff check</code> , <code>ruff format --check</code> e <code>pytest</code> em Python 3.11 e 3.12

S1-02 · ADR de arquitetura

As decisões que guiam todo o projeto ficaram registradas num único documento, `docs/adr/0001-arquitetura.md`, ainda com status "Proposto". Este card não produziu código.

O que o ADR fixa:

- **Estrutura de pacotes:** a árvore final do repositório, com o que entra em cada semana.
- **Convenções:** unidades SI, tempo UTC, referenciais ECI e ECEF, shapes dos arrays (`(N, 3)` por satélite, `(P, N, 3)` por constelação) e a notação dos PDFs (ψ = anomalia média, ℓ = momento angular, h = altitude).
- **Interfaces:** as assinaturas de `Propagator`, `Satellite`, `Constellation` e `GroundStation`. Só `Propagator` foi implementado nesta semana.
- **Fluxo de dados:** propagador → ECI → rotação pelo tempo sideral → ECEF → observador → elevação → visibilidade.
- **Interface gráfica:** os requisitos R1–R10, o layout da tela, o contrato JSON da cena e a escolha de stack (`globe.gl` + `canvas 2D`, comparada com `CesiumJS`, `Plotly`, `PyVista` e `Streamlit`).
- **Simplificações assumidas:** sem precessão e nutação, $UT1 \approx UTC$ (até 0,4 km de erro no equador), segundos intercalares ignorados.

S1-03 · Constantes e unidades

Todas as constantes físicas do projeto vivem num só lugar, com valor idêntico ao dos PDFs e a fonte citada.

Arquivo	O que faz
<code>src/satsim/constants.py</code>	Define μ , $R_\oplus$, $\omega_\oplus$, J_2 , J_3 , J_4 e o ano trópico; deriva a taxa heliossíncrona <code>OMEGA_SUN</code> (0,985647°/dia); traz os auxiliares <code>SECONDS_PER_DAY</code> , <code>DEG2RAD</code> e <code>RAD2DEG</code>
<code>tests/test_constants.py</code>	Confere cada valor contra a tabela dos PDFs e as relações derivadas (por exemplo, a rotação da Terra é cerca de 366 vezes a taxa heliossíncrona)

O módulo documenta uma armadilha comum: modelos como o EGM2008 publicam o coeficiente normalizado $C_{20} = -4,8417e-4$. Usá-lo no lugar de J_2 erra por um fator $\sqrt{5} \approx 2,24$, e a inclinação heliossíncrona sai completamente errada.

S1-04 · Elementos orbitais ↔ estado cartesiano

O simulador converte uma órbita entre suas duas descrições: os 6 elementos orbitais (a , e , i , Ω , ω , ψ), que têm sentido geométrico e são como a missão é especificada, e a posição e velocidade (r , v) no referencial ECI, que é o que a física e o desenho usam. Ida e volta erram menos de $1e-9$ m.

Arquivo	O que faz
<code>src/satsim/elements.py</code>	<code>KeplerianElements</code> (os 6 elementos); <code>solve_kepler</code> (equação de Kepler $E - e \sin E = \psi$ por Newton-Raphson); conversões entre anomalia média, excêntrica e verdadeira; <code>elements_to_state</code> e <code>state_to_elements</code>
<code>tests/test_elements.py</code>	Resíduo de Kepler $< 1e-12$ até $e = 0,95$, ida e volta na órbita de projeto, invariantes (energia, momento angular), latitude máxima de $82,56^\circ$ e vetorização

Tudo é vetorizado: um array de N instantes gera N posições de uma vez. Os casos em que um ângulo deixa de existir são tratados por convenção: órbita circular (sem perigeu, $\omega = 0$) e órbita equatorial (sem linha dos nós, $\Omega = 0$).

S1-05 · KeplerPropagator vetorizado

O primeiro propagador calcula a posição do satélite em qualquer instante pela órbita kepleriana pura (sem perturbações). Ele bate com uma integração numérica de 1 dia a menos de 0,02 mm.

Arquivo	O que faz
<code>src/satsim/propagators/base.py</code>	Classe abstrata <code>Propagator</code> : o contrato que todo propagador cumpre (<code>epoch</code> e <code>propagate(t) → (r, v)</code> com shape <code>(N, 3)</code>); helper <code>as_time_array</code>
<code>src/satsim/propagators/kepler.py</code>	<code>KeplerPropagator</code> : avança a anomalia média, $\psi(t) = \psi_0 + n \cdot t$, e converte em posição com <code>elements_to_state</code> ; expõe <code>mean_motion</code> e <code>period</code>
<code>src/satsim/propagators/__init__.py</code>	Exporta <code>Propagator</code> e <code>KeplerPropagator</code>
<code>tests/test_kepler_propagator.py</code>	Conservação de energia e momento angular em 30 dias, período de 5689,30 s na órbita de projeto, retorno ao estado inicial após um período, comparação com <code>scipy.integrate.solve_ivp</code>

A ideia central: sob gravidade central, cinco elementos ficam constantes e só ψ avança no tempo. Propagar é "elementos $\rightarrow$ avançar $\psi \rightarrow$ posição". Os próximos propagadores (J2, Cowell, SGP4) seguem a mesma interface.

S1-06 · Módulo de tempo (UTC $\leftrightarrow$ JD $\leftrightarrow$ GMST)

O módulo de tempo liga a data civil ao quanto a Terra já girou, que é o que permite saber sobre qual ponto do planeta o satélite está. O tempo sideral calculado bate com o Skyfield a menos de 0,007 s.

Dois referenciais aparecem aqui:

- **ECI** (inercial): eixos fixos em relação às estrelas. É onde o propagador calcula a órbita.
- **ECEF** (fixo na Terra): eixos que giram com o planeta, com x apontando para Greenwich. É onde estão latitude, longitude e a estação de solo.

A passagem entre eles é uma rotação em torno do eixo polar pelo tempo sideral de Greenwich (GMST).

Arquivo	O que faz
<code>src/satsim/astrotime.py</code>	<code>datetime_to_jd</code> e <code>jd_to_datetime</code> (data UTC ↔ Data Juliana); <code>jd_from_epoch</code> (segundos do propagador → JD); <code>gmst</code> (fórmula IAU-82 de Vallado); <code>gmst_rate</code> (derivada analítica do GMST, acrescentada no S1-08)
<code>tests/test_astrotime.py</code>	Datas de referência (J2000, época Unix), fuso -03:00, Exemplo 3-5 de Vallado, taxa de 360,9856°/dia e comparação com o Skyfield em 200 datas entre 1990 e 2040

O nome `astrotime.py` (e não `time.py`) evita conflito com o módulo `time` da biblioteca padrão.

S1-07 · Testes de regressão numérica (golden tests)

Todos os números de referência dos PDFs teóricos estão num único arquivo versionado, com a fonte de cada um, para os testes das próximas semanas reutilizarem. Testes de consistência cruzam esses números entre si para pegar erros de transcrição.

Arquivo	O que faz
<code>tests/golden/reference_values.py</code>	Constantes, órbita de projeto 91/6, Tabelas 1, 3 e 4 do PDF da Semana 4, taxas seculares de J2 e valores de arrasto do PDF da Semana 3, em classes congeladas, com a unidade no nome (<code>h_km</code> , <code>i_deg</code>)
<code>tests/test_golden_consistency.py</code>	Recalcula valores a partir de outros (por exemplo, 86400 / período nodal ≈ revoluções por dia) e confere que o mesmo valor é igual em tabelas diferentes

Os testes acharam uma diferença de arredondamento no próprio PDF: as altitudes mínima e máxima da órbita de projeto (502,5 e 517,4 km) não batem com os elementos (502,55 e 517,35 km). O valor do PDF foi mantido e o teste ficou marcado como falha esperada (`xfail`), com a explicação.

S1-08 · Protótipo visual: globo 3D + traço no solo 2D

A primeira versão visível do simulador é uma página única, que abre com duplo clique e sem internet. Ela mostra, sincronizados, um globo 3D com os satélites em movimento e um mapa 2D com o traço no solo, a 60 quadros por segundo com até 18 satélites.

Arquivo	O que faz
<code>src/satsim/app/scene_export.py</code>	<code>build_scene</code> : propaga cada satélite numa grade de tempo, gira ECI → ECEF pelo GMST, calcula latitude, longitude e altitude e devolve a cena no formato JSON do ADR; classes provisórias <code>SceneSatellite</code> e <code>SceneStation</code>
<code>src/satsim/app/static/viewer.html</code> , <code>viewer.css</code> , <code>viewer.js</code>	O viewer: tema escuro, globo, mapa, legenda e controles. Só lê a cena, interpola entre amostras e desenha; não há equação orbital em JavaScript
<code>src/satsim/app/static/vendor/globe.gl.min.js</code>	Biblioteca globe.gl 2.46.2 (MIT), que inclui o three.js, versionada para funcionar offline
<code>src/satsim/app/static/textures/earth_2048.jpg</code>	Textura da Terra NASA Blue Marble (domínio público), 2048 × 1024
<code>scripts/build_demo.py</code>	Monta a cena (órbita heliossíncrona de projeto + um exemplo kepleriano com $i = 51,64^\circ$, janela de 6 h, passo de 10 s) e embute tudo em <code>demo/index.html</code>
<code>docs/creditos.md</code>	Fonte, versão e licença de cada asset, e a verificação de uso offline
<code>tests/test_scene_export.py</code>	Estrutura do JSON, latitude máxima de $82,56^\circ$, consistência entre posição e subponto, 1 a 3 satélites e continuidade da longitude no antimeridiano

Nesta fase a vista mantém a Terra fixa e o subponto é calculado com a Terra esférica; o modelo WGS84 entra na Semana 2. A órbita do exemplo com $i = 51,64^\circ$ não é a ISS real, que chega na Semana 3 via TLE.

Como rodar a simulação do S1-08

Com Python 3.11 ou mais recente, os comandos abaixo instalam o projeto, rodam os testes e geram a demo, a partir da raiz do repositório:

```
git clone
git@github.com:RenatodaCostaSantos/satellite_constellation_simulation.git
cd satellite_constellation_simulation
git checkout semana-1

python -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -e ".[dev]"

pytest                             # 118 testes passando, 1 falha esperada
python scripts/build_demo.py        # gera demo/index.html (~2,5 MB)
xdg-open demo/index.html            # Linux; no macOS: open; ou duplo clique
no arquivo
```

A demo não precisa de servidor nem de internet. Para adicionar um satélite, acrescente uma linha à lista `SATELLITES` em `scripts/build_demo.py` e gere de novo:

```
kepler_sat("kep2", "Terceiro (i = 30°)", "#b26bff", a_km=7200.0, e=0.01,
i_deg=30.0, raan_deg=200.0, argp_deg=0.0, psi_deg=90.0),
```

Controle	Ação
▶ / ■■ (ou barra de espaço)	Reproduzir ou pausar
1x · 10x · 60x · 300x · 1000x	Velocidade da simulação
Linha do tempo	Arrastar para qualquer instante da janela de 6 h
⏮ início	Voltar ao início da janela
Caixas na lista de satélites	Mostrar ou ocultar cada satélite
Arrastar ou roda do mouse no globo	Girar ou dar zoom